**Tugas Individu : Mata Kuliah Algoritma dan Struktur Data**

# Laporan Praktikum Analisis Konvolusi Citra dan Optimasi Strassen

Disusun Oleh :

**Ahmad Zul Taufik ( D121251028 )**

**PROGRAM STUDI TEKNIK INFORMATIKA**

**FAKULTAS TEKNIK**

**UNIVERSITAS HASANUDDIN**

**2026**

## Sinkronisasi Konsep Dasar

Bagian ini membahas fondasi dasar terkait memori dan struktur data yang menjadi kunci dalam pemrosesan citra. Penjelasan diberikan secara konseptual sekaligus dikaitkan dengan implementasi.

### 1. Variabel dan Tipe Data

**Pertanyaan:** Jika memori adalah sebuah gudang besar berisi kotak-kotak, apa perbedaan antara kotak berlabel int, float, dan unsigned char?

**Jawaban:**
Jika memori dianalogikan sebagai gudang, maka tipe data adalah jenis kotak dengan ukuran dan aturan isi yang berbeda.

- **int** dapat dianggap sebagai kotak berukuran 4 byte yang menyimpan bilangan bulat, baik positif maupun negatif.
- **float** juga berukuran 4 byte, tetapi digunakan untuk menyimpan bilangan pecahan dengan format khusus (IEEE 754).
- **unsigned char** adalah kotak kecil berukuran 1 byte yang hanya menyimpan nilai 0 sampai 255.

**Pertanyaan:** Mengapa untuk data warna piksel (0–255) kita lebih memilih unsigned char daripada int?

**Jawaban:**
Karena nilai warna hanya berada di rentang 0–255, penggunaan **unsigned char** jauh lebih efisien. Jika menggunakan **int**, sebagian besar memori tidak terpakai. Pada gambar berukuran besar, hal ini bisa menyebabkan pemborosan memori yang signifikan.

**2. Array 1D vs 2D**

**Pertanyaan:** Mengapa indeks piksel (x, y) dihitung dengan rumus indeks = y × width + x?

**Jawaban:**
Walaupun gambar terlihat seperti matriks 2D, data sebenarnya disimpan secara linear di memori. Penyimpanan dilakukan per baris (*row-major*).

Untuk mencapai posisi (x, y):

- Kita melewati y baris sebelumnya → y × width
- Lalu geser ke posisi x pada baris tersebut

Dalam program dengan channel warna, rumusnya menjadi: **((y × width) + x) × c + ch**

**Pertanyaan:** Apa yang terjadi jika kita mengakses A[100][100] pada array 100×100?

**Jawaban:**
Karena indeks dimulai dari 0, elemen terakhir adalah A[99][99]. Mengakses A[100][100] berarti keluar dari batas array (*out of bounds*), yang bisa menyebabkan error serius seperti *segmentation fault* atau menghasilkan data acak.

**3. Variabel Lokal vs Global**

**Pertanyaan:** Apakah variabel sum dalam fungsi apply_blur bisa diakses dari main?

**Jawaban:**
Tidak bisa. Variabel tersebut bersifat lokal, sehingga hanya berlaku di dalam fungsi tempat ia dideklarasikan.

**Pertanyaan:** Apa yang terjadi pada variabel sum saat fungsi selesai dieksekusi?

**Jawaban:**
Variabel lokal disimpan di stack. Ketika fungsi selesai, stack pointer kembali ke posisi semula, sehingga memori yang digunakan variabel tersebut otomatis dilepaskan dan dapat digunakan kembali.

**4. Struktur Kontrol (Loop)**

**Pertanyaan:** Apa fungsi dari loop bersarang pada apply_blur?

**Jawaban:**
Loop digunakan untuk memproses data dari tingkat global ke detail:

- Loop **y**: menjelajahi baris gambar
- Loop **x**: menjelajahi kolom pada tiap baris
- Loop **ch**: memproses tiap channel warna
- Loop **ky** dan **kx**: mengakses area kernel untuk menghitung nilai blur

## Pertanyaan Analisis

### 1. Boundary Check

**Pertanyaan:** Apa yang terjadi jika tidak ada pengecekan batas dalam konvolusi?

**Jawaban:**
Tanpa pengecekan batas, program bisa mengakses memori di luar gambar. Hal ini dapat menyebabkan noise (warna acak) atau bahkan crash (*segmentation fault*).

Contoh pengecekan:

```
if (px >= 0 && px < w && py >= 0 && py < h)
```

### 2. Kompleksitas Waktu

**Pertanyaan:** Mengapa kompleksitas algoritma adalah $O(N^2 \times K^2)$?

**Jawaban:**
Karena:

- Terdapat $N^2$ piksel dalam gambar
- Setiap piksel membutuhkan $K^2$ operasi dari kernel

Semakin besar ukuran gambar dan kernel, semakin besar pula waktu komputasi yang dibutuhkan.

### 3. Optimasi Algoritma

**Pertanyaan:** Bagaimana cara mempercepat algoritma dengan separable convolution?

**Jawaban:**
Kernel 2D dapat dipecah menjadi dua kernel 1D: horizontal dan vertikal.

Proses dilakukan dalam dua tahap:

- Blur horizontal
- Blur vertikal

Dengan cara ini, kompleksitas berubah dari :

$O(N^2 \times K^2) \rightarrow O(N^2 \times 2K)$

Hasilnya jauh lebih efisien.

## Solusi Modul 2: Alokasi Memori Dinamis

**Pertanyaan:** Mengapa perlu menggunakan malloc untuk array 2D (alloc_m)?

**Jawaban:**
Array besar tidak aman jika disimpan di stack karena kapasitasnya terbatas. Oleh karena itu, digunakan heap dengan malloc.

Langkahnya:

- Membuat array pointer:

```
int** m = (int**) malloc(n * sizeof(int*));
```

- Mengalokasikan tiap baris:

```
for (int i = 0; i < n; i++) {
    m[i] = (int*) malloc(n * sizeof(int));
}
```

Pendekatan ini memungkinkan penggunaan memori yang lebih fleksibel. Namun, memori harus dibebaskan dengan free() untuk menghindari *memory leak*.

Penggunaan time.h biasanya untuk mengisi data secara acak dengan rand(), yang berguna untuk pengujian performa.